Rapport d'analyse et de correction

Non-persistance de l'entité **VulnInfo** dans le pipeline d'ingéstion XML Qualys
Projet *scr_pfa* — module backend/app (Spring Boot / Spring Batch / JPA)
17 août 2026

Symptôme	La table VULN_INFO reste vide après exécution du job d'ingéstion
Périmètre	Hostdto, HostMapper, OsTypeProcessor, BatchConfig
Causes identifiées	4 défauts cumulatifs (2 bloquants indépendamment)
État	Correctifs appliqués — compilation validée

1. Contexte technique

Le module `backend/app` ingère des rapports de vulnérabilité Qualys au format XML vers une base MySQL, via un job Spring Batch. La chaîne de traitement est la suivante :

```
Fichier XML                                    (rapport Qualys)
     |
     v
StaxEventItemReader + Jaxb2Marshaller          fragment racine = <HOST>
     |  produit un Hostdto par hote
     v
OsTypeProcessor (MapStruct)                    Hostdto  ->  Host
     |
     v
JpaItemWriter                                  entityManager.merge(...)
     |
     v
MySQL : tables HOST et VULN_INFO
```

Le modèle de données associe un hôte à ses vulnérabilités par une relation bidirectionnelle `@OneToMany` / `@ManyToOne`. La persistance des `VulnInfo` repose donc entièrement sur la cascade déclarée côté `Host` :

```
// entity/Host.java
@OneToMany(mappedBy = "host", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private List<VulnInfo> vulns = new ArrayList<>();
```

Ce point est essentiel pour la suite : **aucun writer ne persiste directement les VulnInfo**. Si la liste `vulns` de l'objet `Host` transmis au writer est vide, la cascade n'a rien à propager et aucune ligne n'est insérée — sans la moindre erreur, ni exception, ni log. C'est précisément ce qui rendait le bug silencieux.

2. Synthèse du diagnostic

Quatre défauts se cumulent sur le trajet des données. Les deux premiers suffisent chacun à vider intégralement le résultat ; les deux suivants auraient produit des données incorrectes une fois les premiers corrigés.

N°	Étape	Défaut	Conséquence
1	Lecture XML (JAXB)	Wrapper `VULN_INFO_LIST` non déclaré	Bloquant : liste toujours `null`
2	Mapping (MapStruct)	Noms de propriétés différents, sans `@Mapping`	Bloquant : `vulns` jamais renseigné
3	Mapping (relation)	Back-reference `VulnInfo.host` non initialisée	Clé étrangère `host_id` à NULL
4	Écriture (Batch)	Classifier de writers incohérent avec le processor	Routage aléatoire, masqué par l'effacement de type

3. Analyse détaillée des causes

3.1 — JAXB ne remplissait jamais la liste : wrapper manquant

Dans le rapport Qualys, les éléments <VULN_INFO> ne sont pas des enfants directs de <HOST> : ils sont regroupés dans un conteneur intermédiaire <VULN_INFO_LIST>.

```
<HOST>
  <IP>64.41.200.243</IP>
  <DNS>demo13.s02.sjc01.qualys.com</DNS>
  <OPERATING_SYSTEM>Ubuntu / Tiny Core Linux / Linux 2.6.x</OPERATING_SYSTEM>
  <VULN_INFO_LIST>            <-- conteneur intermediaire
    <VULN_INFO>
      <QID id="qid_6">6</QID>
      <TYPE>Ig</TYPE>
      ...
```

Or le DTO déclarait la collection directement au niveau de l'hôte :

```
// AVANT -- dto/Hostdto.java
@XmlElement(name = "VULN_INFO")
private List<VulnInfoDto> vulnInfoDtos;
```

Avec cette déclaration, JAXB recherche des éléments `VULN_INFO` parmi les enfants immédiats de `HOST`. Il n'en trouve aucun, ignore silencieusement le sous-arbre `VULN_INFO_LIST` qu'il ne sait pas rattacher, et laisse le champ à `null`. **Dès la lecture, l'intégralité des vulnérabilités était perdue.** Aucun avertissement n'est émis : le comportement par défaut de JAXB est de tolérer les éléments inconnus.

3.2 — MapStruct ne mappait pas la collection : noms divergents

MapStruct associe les propriétés source et cible **par correspondance de nom**. Ici la source s'appelle `vulnInfoDtos` et la cible `vulns`. Sans annotation `@Mapping` explicite, MapStruct ne peut pas établir le lien et se contente d'un avertissement de compilation.

```
// AVANT -- mapper/HostMapper.java
@Mapper(componentModel = "spring", uses = OsAdapterUtil.class)
public interface HostMapper {
    Host toEntity(Hostdto hostdto);
}
```

La preuve se lisait directement dans l'implémentation générée sous `target/generated-sources` : la ligne `setVulns(...)` était tout simplement absente.

```
// AVANT -- HostMapperImpl.java (genere)
Host host = new Host();

host.setId( hostdto.getId() );
host.setIp( hostdto.getIp() );
host.setDns( hostdto.getDns() );
host.setOs( osAdapterUtil.classify( hostdto.getOs() ) );
                                  <-- aucun appel a setVulns(...)
return host;
```

L'objet `Host` transmis au writer conservait donc sa liste vide d'initialisation. La cascade `CascadeType.ALL` parcourait une collection de taille zéro : rien à persister. **Ce défaut aurait suffi à lui seul, même si le problème JAXB n'avait pas existé.**

3.3 — Relation bidirectionnelle : côté propriétaire non renseigné

L'association est déclarée `mappedBy = "host"`. En JPA, cela signifie que **le côté propriétaire de la relation est** `VulnInfo.host`, et non la collection côté `Host`. C'est ce champ, et lui seul, qui détermine la valeur écrite dans la colonne de clé étrangère.

Ajouter un élément à `host.getVulns()` sans positionner `vuln.setHost(host)` produit donc des lignes orphelines avec `host_id = NULL`. Le mapping généré par MapStruct ne pouvait pas deviner cette réciprocité : le DTO `VulnInfoDto` ne possède aucun champ pointant vers l'hôte. Il fallait donc l'établir explicitement après le mapping.

3.4 — Aiguillage des writers incohérent avec le processor

Le reader ne produit qu'un seul type de fragment (`fragmentRootElementNames = {"HOST"}`), donc uniquement des `Hostdto`. La branche `item instanceof VulnInfoDto` du processor était du **code mort** : elle n'a jamais pu s'exécuter.

Plus grave, le classifier testait un type que le processor avait déjà transformé :

```
// AVANT -- config/BatchConfig.java
Classifier<Object, ItemWriter<? super Object>> classifier = item -> {
    if (item instanceof Hostdto)  return (ItemWriter) hostWriter;   // jamais vrai :
    if (item instanceof VulnInfo) return (ItemWriter) vulnWriter;   // le processor a
    return (ItemWriter) qidWriter;                                  // deja rendu un Host
};
```

Le processor renvoie un `Host` (entité), pas un `Hostdto`. Aucune des deux conditions n'était satisfaite et **tous** les éléments tombaient dans la branche par défaut, le `qidWriter`. Cela ne provoquait aucune erreur visible : à l'exécution, l'effacement de type (*type erasure*) fait qu'un `JpaItemWriter<Qiddto>` appelle `entityManager.merge()` sur n'importe quel objet. Les hôtes étaient donc bien écrits — **par accident** — ce qui a contribué à masquer l'anomalie et à faire croire que seule la partie VulnInfo était défaillante.

4. Correctifs appliqués

4.1 — Déclaration du wrapper XML

Fichier : `dto/Hostdto.java`

```
// APRES
@XmlElementWrapper(name = "VULN_INFO_LIST")
@XmlElement(name = "VULN_INFO")
private List<VulnInfoDto> vulnInfoDtos;
```

`@XmlElementWrapper` indique à JAXB que la collection est encapsulée dans un élément intermédiaire. La liste est désormais correctement peuplée à la lecture.

4.2 et 4.3 — Mapping de la collection et de la back-reference

Fichier : `mapper/HostMapper.java`

```
// APRES
@Mapper(componentModel = "spring", uses = {OsAdapterUtil.class, VulnMapper.class})
public interface HostMapper {

    @Mapping(target = "vulns", source = "vulnInfoDtos")
    Host toEntity(Hostdto hostdto);

    @AfterMapping
    default void linkVulns(@MappingTarget Host host) {
        if (host.getVulns() == null) {
            host.setVulns(new ArrayList<>());
            return;
        }
        host.getVulns().forEach(vuln -> vuln.setHost(host));
    }
}
```

Trois éléments dans ce correctif :

- `@Mapping(target = "vulns", source = "vulnInfoDtos")` établit explicitement la correspondance entre les deux noms divergents.
- `uses = VulnMapper.class` fait réutiliser le mapper existant pour convertir chaque élément, plutôt que de générer un mapping anonyme dupliqué.
- `@AfterMapping` positionne la back-reference sur chaque enfant, ce qui garantit une clé étrangère valide. Le cas `null` est ramené à une liste vide, car un hôte sans `VULN_INFO_LIST` est un cas légitime du format.

4.4 — Typage explicite du pipeline batch

Le classifier et ses writers superflus ont été supprimés au profit d'un pipeline typé de bout en bout. Puisque la persistance des `VulnInfo` passe par la cascade depuis `Host`, un writer unique suffit.

Fichier : `service/OsTypeProcessor.java`

```
// APRES
public class OsTypeProcessor implements ItemProcessor<Hostdto, Host> {
    private final HostMapper hostMapper;

    @Override
    public @Nullable Host process(Hostdto item) {
        return hostMapper.toEntity(item);
    }
}
```

Fichier : `config/BatchConfig.java`

```
// APRES
return new StepBuilder(jobRepository)
        .<Hostdto, Host>chunk(100)
        .transactionManager(transactionManager)
        .reader(multiResourceItemReader)
        .processor(osTypeProcessor)
        .writer(hostWriter)
        .build();
```

Les readers ont été retypés de `<Object>` vers `<Hostdto>`. Le bénéfice est important : le compilateur vérifie désormais la cohérence entre le reader, le processor et le writer. Les incohérences de type comme celle décrite en 3.4 deviennent des erreurs de compilation au lieu de défaillances silencieuses à l'exécution.

Récapitulatif des fichiers modifiés

Fichier	Modification
dto/Hostdto.java	Ajout de `@XmlElementWrapper("VULN_INFO_LIST")`
mapper/HostMapper.java	Ajout de `@Mapping` vers `vulns`, de `VulnMapper` dans `uses`, et d'un `@AfterMapping` pour la back-reference
service/OsTypeProcessor.java	Typage `ItemProcessor<Hostdto, Host>` ; suppression de la branche morte `VulnInfoDto`
config/BatchConfig.java	Suppression du `ClassifierCompositeItemWriter` et des writers inutilisés ; readers et step typés `<Hostdto, Host>`

5. Vérification effectuée

La compilation du module a été relancée (`mvnw compile`) et se termine sans erreur, ce qui valide notamment le retypage générique du step Spring Batch.

L'implémentation régénérée par MapStruct confirme que les correctifs 4.2 et 4.3 sont bien pris en compte :

```
// APRES -- HostMapperImpl.java (regenere)
@Autowired private OsAdapterUtil osAdapterUtil;
@Autowired private VulnMapper vulnMapper;

@Override
public Host toEntity(Hostdto hostdto) {
    if ( hostdto == null ) return null;

    Host host = new Host();

    host.setVulns( vulnInfoDtoListToVulnInfoList( hostdto.getVulnInfoDtos() ) );
    host.setId( hostdto.getId() );
    host.setIp( hostdto.getIp() );
    host.setDns( hostdto.getDns() );
    host.setOs( osAdapterUtil.classify( hostdto.getOs() ) );

    linkVulns( host );        // positionne la back-reference sur chaque VulnInfo

    return host;
}

protected List<VulnInfo> vulnInfoDtoListToVulnInfoList(List<VulnInfoDto> list) {
    if ( list == null ) return null;
    List<VulnInfo> list1 = new ArrayList<VulnInfo>( list.size() );
    for ( VulnInfoDto vulnInfoDto : list ) {
        list1.add( vulnMapper.toEntity( vulnInfoDto ) );
    }
    return list1;
}
```

Les appels à `setVulns(...)` et à `linkVulns(host)`, absents de la version précédente, sont désormais présents.

Limite de la vérification

À ce stade, la validation porte sur la compilation et sur le code généré. **Le job n'a pas été exécuté de bout en bout contre la base MySQL.** La correction du wrapper JAXB (section 3.1) en particulier ne peut être confirmée qu'à l'exécution, puisqu'elle relève d'un comportement d'*unmarshalling* et non d'une garantie du compilateur.

6. Recommandations

- **Valider par une exécution réelle.** Lancer le job sur `Qualys_Sample_Report.xml` et vérifier en base que la table VULN_INFO est peuplée et que la colonne `host_id` n'est jamais NULL.
- **Ajouter un test de non-régression sur la lecture XML.** Un test qui désérialise un fragment `<HOST>` et assure que la liste des vulnérabilités n'est pas vide protégerait contre le retour du défaut 3.1, qui est le seul à échapper entièrement au compilateur.
- **Traiter les avertissements MapStruct comme des erreurs.** Le défaut 3.2 avait généré un avertissement de compilation ignoré. L'option `unmappedTargetPolicy = ReportingPolicy.ERROR` aurait fait échouer la construction et rendu le problème immédiat.

- **Ne pas revenir à des pipelines typés `Object`.** C'est ce typage permissif qui a permis au défaut 3.4 de passer inaperçu pendant toute la durée du problème.

7. Conclusion

Le problème ne provenait pas de la configuration JPA ni de la base de données : le schéma était correct et la cascade correctement déclarée. Les vulnérabilités étaient perdues **avant même d'atteindre la couche de persistance**, d'abord à la désérialisation XML, puis au mapping objet. Chaque maillon défaillant était par nature silencieux — JAXB ignore les éléments inconnus, MapStruct se contente d'un avertissement, et une cascade sur collection vide est une opération parfaitement légitime — ce qui explique l'absence totale de trace d'erreur malgré une perte de données complète.